

React Lifecycle, VDOM Diffing, Create-react-app, Hooks

RUJUAN XING

Component Lifecycle

REACT

Problem Intro

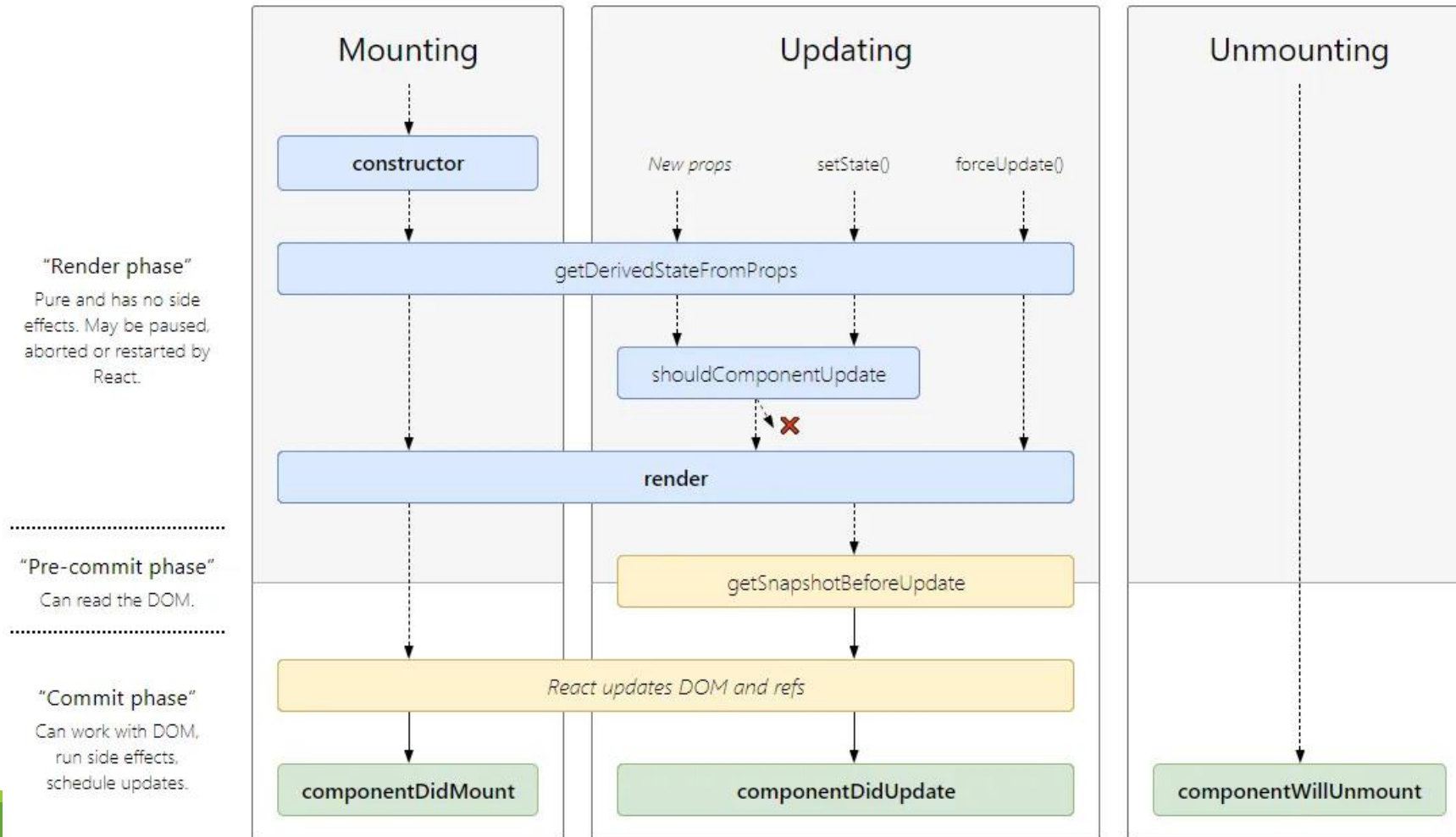
Open the browser console and check how rapidly “render...” is printed.

Why?

```
class MyComponent extends React.Component {  
  
  state = {opacity: 1};  
  
  render() {  
    console.log('render...');  
    setInterval(() => {  
      const {opacity} = this.state;  
      if(opacity <= 0.1){  
        this.setState({opacity: 1});  
      } else {  
        this.setState({opacity: opacity-0.1});  
      }  
    }, 200);  
    return (  
      <div>  
        <h1 style={this.state}>Time flies fast!</h1>  
        <button onClick={() => clearInterval(this.state.timerId)}>Stop</button>  
      </div>  
    );  
  }  
}
```

Component Lifecycle

class components have a lifecycle that consists of various methods that are automatically called at different stages of the component's existence. These methods allow you to perform actions such as initializing state, rendering content, handling updates, and cleaning up resources.



Lifecycle Methods

1. `constructor(props)` : This method is called when the component is being initialized. It's where you typically set the initial state and bind event handlers. Always start with `super(props)` to call the constructor of the parent class.
2. `render()` : The render method is responsible for returning the JSX that represents the component's UI. It should be a pure function and not cause any side effects.
3. `componentDidMount()` : This method is called after the component is mounted in the DOM. It's an ideal place to initiate network requests, set up subscriptions, or perform other side effects.
4. `componentDidUpdate(prevProps, prevState)` : This method is called after the component is re-rendered due to state or prop changes. It's a good place to interact with the updated DOM or perform side effects.
5. `componentWillUnmount()` : This method is called just before the component is removed from the DOM. You can use it to clean up resources, such as canceling network requests or unsubscribing from subscriptions.

Lifecycle Methods (cont.) – rare use

- 6. `shouldComponentUpdate(nextProps, nextState)` : This method is called before rendering when new props or state are received. You can use it to control whether the component should re-render or not.
- 7. `static getDerivedStateFromProps(props, state)` : It's called when a component is about to receive new props, either as it's being mounted or as its parent component is re-rendering. The main purpose of this method is to update the component's state based on changes in the incoming props.
- 8. `getSnapshotBeforeUpdate(prevProps, prevState)` : It enables your component to capture some information from the DOM (e.g. scroll position) before it is potentially changed. Any value returned by this lifecycle method will be passed as a parameter to `componentDidUpdate`.

Lifecycle Demo

```
<script type="text/babel">
class Count extends React.Component {
  state = {counter: 0};

  increase = () => {
    const {counter} = this.state;
    this.setState({counter: counter + 1});
  }

  unmount = () => {
    root.unmount();
  }

  constructor(props) {
    super(props);
    console.log('count --- constructor');
  }

  componentDidMount() {
    console.log('count --- componentDidMount');
  }

  shouldComponentUpdate(nextProps, nextState) {
    return true;
  }
}
```

```
componentDidUpdate(prevProps, prevState) {
  console.log('count --- componentDidUpdate');
}

componentWillUnmount() {
  console.log('count --- componentWillUnmount');
}

static getDerivedStateFromProps(props, state) {
  console.log('count --- getDerivedStateFromProps', props, state);
  return {counter: state.counter + props.increStep};
}

render() {
  console.log('count --- render');
  return (
    <div>
      <h1>Current Number: {this.state.counter}</h1>
      <button onClick={this.increase}>Increase</button>
      <button onClick={this.unmount}>Unmount</button>
    </div>
  );
}

const root = ReactDOM.createRoot(document.getElementById('container'));
root.render(<Count username="Tina" increStep={2}/>);

</script>
```

Virtual DOM Diffing

REACT

Demonstrating the Existence of React's Virtual DOM Diffing

In the Clock demo on the right-hand side.

When we input a value into the text input field, the `render()` method is triggered after one second, resulting in the update of only the time statement within the `<p>` tag, while the input fields remain unaffected.

If Virtual DOM Diffing were not in place, all elements, including the input fields, would be updated.

```
<script type="text/babel">
  class Clock extends React.Component {

    state = {time: new Date()}

    componentDidMount(){
      setInterval(() => {
        this.setState({time: new Date()});
      }, 1000);
    }

    render(){
      return (
        <div>
          <h1>Welcome to DOM Diffing</h1>
          <input type="text" />
          <p>Current Time is {this.state.time.toLocaleTimeString()}<input type="text" /></p>
        </div>
      );
    }
  }

  const root = ReactDOM.createRoot(document.getElementById('container'));
  root.render(<Clock />)
</script>
```

Virtual DOM Diffing

In React, the Virtual DOM (VDOM) diffing is a core concept that contributes to the framework's efficiency and performance. The VDOM is a lightweight copy of the actual DOM, and React uses it to compare the current representation of a component with the next one to determine what changes need to be made.

What's the usage of "key" attribute in virtual DOM?

In React, the "key" attribute is used to help React identify and track individual elements in a list when rendering or updating components efficiently. It is not part of the Virtual DOM (VDOM) itself but plays a crucial role in how React reconciles changes in the real DOM using the VDOM.

How Diffing Algorithm Works?

When data in the state changes, React generates a new virtual DOM based on the updated data. Following this, React compares the new virtual DOM with the old virtual DOM using a diffing process.

Rules:

1. The same key as the new virtual DOM **was found** in the old virtual DOM
 - 1) If the content in the virtual DOM hasn't changed, the previous real DOM is used directly.
 - 2) If the content in the virtual DOM has changed, a new real DOM is generated and then replaces the previous real DOM on the page.
2. The same key as the new virtual DOM **was NOT found** in the old virtual DOM
 - 1) Create a new real DOM based on the data, and then render it on the page.

Any problem of using index as key?

In the Person demo on the right-hand side.

We add a new Person at the end of the array, then change the state.

What will occur if we instead insert the person at the beginning of the array?

```
class Person extends React.Component {  
  
  state = { persons: [ {id: 1, name: 'John', age: 18}, {id: 2, name: 'Edward', age: 19}] };  
  
  add = () => {  
    const {persons} = this.state;  
    const id = persons.length + 1;  
    const newP = { id, name: 'JJ', age: 20 };  
    this.setState({persons: [...persons, newP]})  
  }  
  
  render() {  
    return (  
      <div>  
        <ul>  
          {  
            this.state.persons.map((p, index) => <li key={index}>{p.name}, {p.age}</li>)  
          }  
        </ul>  
        <button onClick={this.add}>Add a new Person</button>  
      </div>  
    );  
  }  
}
```

The Issues That May Arise When Using Index as a Key

1. If operations on the data, such as reverse addition, reverse deletion, or any actions disrupting the order, are performed, unnecessary updates to the real DOM may occur. While the user interface remains unaffected, this can lead to low efficiency.
 - If the structure of JSX also includes DOM elements like input fields, it may lead to incorrect DOM updates, causing issues in the user interface.
2. If there's no operation like reverse addition to the data, there's no problem to use index as key.

Index as key issue - Demo

Add newP in the front of the array

An input field in the structure of JSX. VDOM diffing only compares the type of input field, the value doesn't compare so real DOM is reused which cause doesn't match.

```
class Person extends React.Component {  
  
  state = {  
    persons: [  
      {id: 1, name: 'John', age: 18},  
      {id: 2, name: 'Edward', age: 19}  
    ]  
  };  
  
  add = () => {  
    const {persons} = this.state;  
    const id = persons.length + 1;  
    const newP = { id, name: 'JJ', age: 20 };  
    this.setState({persons: [newP, ...persons]});  
  }  
  
  render() {  
    return (  
      <div>  
        <ul>  
          {  
            this.state.persons.map((p, index) => <li key={index}>{p.name}, {p.age} <input type="text" /></li>)  
          }  
        </ul>  
        <button onClick={this.add}>Add a new Person</button>  
      </div>  
    );  
  }  
}
```

Guidelines for Selecting Keys

1. It's best to use a unique identifier for each piece of data as the key, such as primary key, phone number, student ID, SSN, etc.
2. If you are certain that you're only displaying data, using the index is also acceptable.

```
class Person extends React.Component {  
  //same code as previous slide, not listed here.  
  
  render() {  
    return (  
      <div>  
        <ul>  
          {  
            this.state.persons.map(p => <li key={p.id}>{p.name}, {p.age} <input type="text" /></li>  
          }  
        </ul>  
        <button onClick={this.add}>Add a new Person</button>  
      </div>  
    );  
  }  
}
```

Create React App

REACT

Create React App

"Create React App" (CRA) is a popular and officially supported tool for quickly setting up and bootstrapping React.js applications. It's an open-source project maintained by Facebook and the React community, designed to simplify the process of starting, configuring, and developing React applications without the need for complex build configurations or setup.

1. **Zero Configuration:** Create React App allows you to create a new React application with zero configuration. It sets up a development environment with all the necessary tools and build configurations, including Webpack, Babel, and ESLint, without requiring you to manually configure them.
2. **Opinionated Defaults:** CRA comes with a set of opinionated defaults and best practices for structuring your project, such as placing components in a src directory and organizing files in a recommended way. These defaults help maintain consistency across React projects.
3. **Built-in Development Server:** It includes a development server that enables hot-reloading, allowing you to see immediate updates as you make changes to your code. No need to manually set up a server or refresh the browser.
4. **Scripts:** CRA provides a set of predefined scripts (e.g., npm start, npm build, npm test) that streamline common development tasks. These scripts handle tasks like starting the development server, building the production version of your app, and running tests.
5. **Configurability:** While CRA aims to be zero-config, it also offers advanced configuration options. You can "eject" from the default configuration to gain full control over the build process if your project requires customizations.
6. **Dependencies:** Create React App manages project dependencies, including React, React DOM, and other commonly used libraries, ensuring that you start with the latest versions.

Install and Start

1. Create a new project
 - `npx create-react-app hello-react`
2. Switch to project folder
 - `cd hello-react`
3. Start project
 - `npm start`

You'll need to have Node ≥ 14 on your local development machine.

`npx` is a command-line tool that comes with `npm` (Node Package Manager) and is used for executing Node.js packages. It's particularly useful for running packages that are installed as dependencies but don't have globally installed command-line tools.

Folder Structure – public folder

public ---- static resources

 favicon.icon ----- favorite icon

index.html ----- main page

 logo192.png ----- logo

 logo512.png ----- logo

 manifest.json ----- plays a crucial role in defining the behavior and appearance of the PWA when users install it on their devices

 robots.txt ----- robots.txt is not specific to React.js but is a standard file used in web development to instruct web crawlers (like search engine bots) on how to interact with a website. It's used to define rules and permissions for web crawlers, specifying which parts of the site they are allowed or disallowed to access and index.

Folder Structure – src folder

src ---- source code

App.css ----- Style of App component

App.js ----- App Component

App.test.js ---- App testing file

index.css ----- Style

index.js ----- Project Entry

logo.svg ----- logo

reportWebVitals.js

--- measure and report key performance metrics that can help developers and site operators analyze and optimize the user experience. (need web-vitals library)

setupTests.js

---- unit tests of Components (need jest-dom library)

Hooks:
useState, useRef,
useCallback, useMemo

REACT

Hooks

Hooks are functions that allow functional components to "hook into" React state and lifecycle features.

There are many hooks:

1. State Hooks
2. Ref Hooks
3. Effect Hooks
4. Context Hooks
5. ...

State Hook - useState

- `useState` is a React Hook that lets you add a state variable to your component.

```
const [state, setState] = useState(initialState);
```

- `initialState`: The value you want the state to be initially. It can be a value of any type, but there is a special behavior for functions. This argument is ignored after the initial render.
 - If you pass a function as `initialState`, it will be treated as an initializer function. It should be pure, should take no arguments, and should return a value of any type. React will call your initializer function when initializing the component, and store its return value as the initial state.
- `useState` returns an array with exactly two values:
 1. The current state. During the first render, it will match the `initialState` you have passed.
 2. The `set` function that lets you update the state to a different value and trigger a re-render.
- The convention is to name state variables like `[something, setSomething]` using array destructuring.

useState Demo

```
import {useState} from "react";

export default function Counter(){

  const [count, setCount] = useState(0);
  function increase(){
    setCount(count + 1);
  }

  return (
    <div>
      <h1>Current Count is: {count}</h1>
      <button onClick={increase}>Click Me</button>
    </div>
  );
}
```

Ref Hook - useRef

`useRef` is a React Hook that lets you reference a value that's not needed for rendering.

```
const ref = useRef(initialValue)
```

- `initialValue`: The value you want the `ref` object's `current` property to be initially. It can be a value of any type. This argument is ignored after the initial render.
- `useRef` returns an object with a single property:
 - `current`: Initially, it's set to the `initialValue` you have passed. You can later set it to something else. If you pass the `ref` object to React as a `ref` attribute to a JSX node, React will set its `current` property.
 - On the next renders, `useRef` will return the same object.

useRef Demo

```
export default function MyComponent() {  
  
  const input1 = useRef(null);  
  const input2 = useRef(null);  
  
  function showData1() {  
    alert(input1.current.value);  
  }  
  
  function showData2() {  
    alert(input2.current.value);  
  }  
  
  return (  
    <div>  
      <input ref={input1}/>  
      <button onClick={showData1}>Click to see input1 data</button>  
      <input ref={input2} onBlur={showData2}/>  
    </div>  
  );  
}
```

Communication between Components

In modern front-end frameworks/libraries, the entire page is divided into multiple smaller components. Component-based architecture makes it easy to develop and maintain an application. During the development, you may come across a situation where you need to share the data with other components.

React provides several methods for handling the component communication, each with its appropriate use cases.

- Parent / Child communication
 - Parent to child communication
 - Child to parent communication
- Context API
- Centralized state with Redux
- Event bus

Parent to Child communication

In React, parent components can communicate to child components using a special property defined by React called Props. Props are read-only and they're passed from parent to child component via HTML attributes.

Example:

Let's say that we have a parent component (`UserList`) that wants to display the user list and it passes the individual user information to the child component (`UserDetails`) to render the user info.

Parent to Child communication Demo

```
export default function UserDetails(props){  
  const {id, name, age} = props;  
  return (  
    <ul>  
      <li>Id: {id}</li>  
      <li>Name: {name}</li>  
      <li>Age: {age}</li>  
    </ul>  
  );  
}
```

```
export default function UserList({userData = []}){  
  
  return (  
    <div>  
      {userData.map(u => <UserDetails key={u.id+u.name} {...u} />)}  
    </div>  
  );  
}
```

```
<UserList userData={userData}/>
```

Child to parent communication:

Data from a child component to parent component can be passed using a callback. We can achieve this by following the below steps.

- Create a callback method in the parent component and pass it to the child using props.
- Child component can call this method using `this.props.[callbackFunctionName]` from the child and pass data as an argument.

Example: Let's say that we want to delete a user from the list and as the child component is responsible for displaying the user info, the delete button is rendered by the child component but the child component doesn't have the complete user list to delete a specific user. So, to achieve it we'll pass a callback function from the parent component and the child component will call that function when we click on the delete button.

Child to parent communication Demo

1. Inside UserList define a handler which will be passed to child component

```
const deleteUserById = (id) => {  
  setUsers(users.filter(u => u.id !== id));  
}
```

2. Pass it to child component using props

```
{users.map(u => <UserDetails key={u.id} {...u} clickHandler={deleteUserById}/>)}
```

3. Update child component to invoke the function when click.

```
const {id, name, age, clickHandler} = props;
```

```
<td onClick={() => clickHandler(id)}>X</td>
```

Callback Hook - `useCallback`

The React `useCallback` Hook returns a memoized callback function.

This allows us to isolate resource intensive functions so that they will not automatically run on every render.

The `useCallback` Hook **only runs when one of its dependencies update** which can improve performance.

```
const cachedFn = useCallback(fn, dependencies)
```

- `fn`: The function value that you want to cache.
- `dependencies`: The list of all reactive values referenced inside of the `fn` code. Reactive values include props, state, and all the variables and functions declared directly inside your component body like `[dep1, dep2, dep3]`. React will compare each dependency with its previous value using the `Object.is` comparison algorithm.
- **Return:**
 - On the initial render, `useCallback` returns the `fn` function you have passed.
 - During subsequent renders, it will either return an already stored `fn` function from the last render (if the dependencies haven't changed), or return the `fn` function you have passed during this render.

useCallback Demo

```
const deleteUserById = useCallback(id => {  
  setUsers(users.filter(u => u.id !== id));  
}, [users]);
```

Memo Hook - useMemo

The React `useMemo` Hook returns a memoized value.

The `useMemo` Hook only runs when one of its dependencies update which can improve performance.

```
const cachedValue = useMemo(calculateValue, dependencies)
```

- `calculateValue`: The function calculating the value that you want to cache. It should be pure, should take no arguments, and should return a value of any type. React will call your function during the initial render. On next renders, React will return the same value again if the dependencies have not changed since the last render. Otherwise, it will call `calculateValue`, return its result, and store it so it can be reused later.
- `dependencies`: The list of all reactive values referenced inside of the `fn` code. Reactive values include props, state, and all the variables and functions declared directly inside your component body like `[dep1, dep2, dep3]`. React will compare each dependency with its previous value using the `Object.is` comparison algorithm.
- Returns
 - On the initial render, `useMemo` returns the result of calling `calculateValue` with no arguments.
 - During next renders, it will either return an already stored value from the last render (if the dependencies haven't changed), or call `calculateValue` again, and return the result that `calculateValue` has returned.

useMemo Demo 1

`useCallback(fn, inputs)` is equivalent to `useMemo(() => fn, inputs)`.

```
const deleteUserById = useCallback(id => {  
  setUsers(users.filter(u => u.id !== id));  
}, [users]);
```



```
const deleteUserById = useMemo(() => id => {  
  setUsers(users.filter(u => u.id !== id));  
}, [users]);
```

useMemo Demo 2

The useMemo Hook can be used to keep expensive, resource intensive functions from needlessly running.

Example: we have an expensive function that runs on every render. When changing the count or adding a todo, you will notice a delay in execution.

```
export default function MemoHook() {
  const [todoList, setTodoList] = useState([]);
  const [count, setCount] = useState(0);

  const expensiveCalculation = useCallback(num => {
    console.log("Calculating...");
    for (let i = 0; i < 1000000000; i++) {
      num += 1;
    }
    return num;
  }, []);

  const calculation = useMemo(() => expensiveCalculation(count), [count]);
  // const calculation = expensiveCalculation(count);
  const addTodo = () => {
    setTodoList([...todoList, 'new List']);
  }
}
```

```
const increaseByOne = () => {
  setCount(count + 1);
}

return (
  <div>
    <h1>Todo List</h1>
    {todoList.map((todo, index) => <p key={index}>{todo}</p>)}
    <button onClick={addTodo}>Add a Todo</button>
    <hr/>
    <h1>Count: {count}
      <button onClick={increaseByOne}>+1</button>
    </h1>
    <hr/>
    <h1>Expensive Calculation</h1>
    Result: {calculation}
  </div>
);
}
```

CSS

REACT

Modularize CSS in React

To support CSS Modules alongside regular stylesheets using the [name].module.css file naming convention.

```
import hello from './Hello.module.css';
export default function Hello(){
  return (
    <div>
      <h1 className={hello.title}>Hello Component!!</h1>
    </div>
  )
}
```

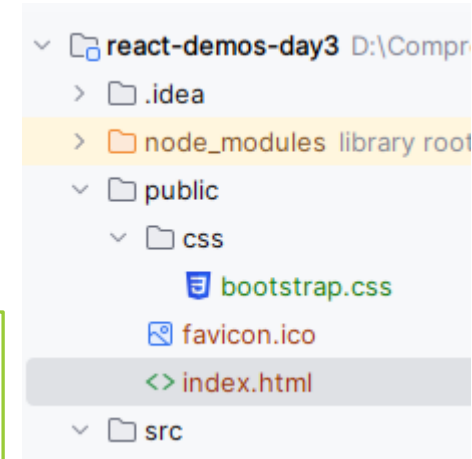
Alternatives:

- Use Less, Sass
- styled-components

Bootstrap

1. Download bootstrap.css
2. Place in `public` -> `css` folder
3. Link in `index.html`

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta
      name="description"
      content="Web site created using create-react-app"
    />
    <link href="css/bootstrap.css" type="text/css" rel="stylesheet" />
    <title>React App</title>
  </head>
  <body>
    <noscript>You need to enable JavaScript to run this app.</noscript>
    <div id="root"></div>
  </body>
</html>
```



Main Point

React Hooks, introduced in React 16.8, are a set of functions that enable developers to add state and side-effect handling to functional components. This innovation allows for the management of component logic without the need for class components. By using hooks like `useState` and `useEffect`, developers can achieve cleaner, more readable code and better component reusability. Hooks have become a cornerstone of modern React development, simplifying state management and making it more approachable for both new and experienced developers.

The existence of a unified field of all the laws of nature, from which everything in the universe originates. It is described as a field of pure consciousness and the source of all knowledge and creativity.